

Desarrollo de Software

PlomerIA

Plataforma de servicios de plomería con IA

Grupo 14

Aldana Benavent · María Florencia Iñiguez Trejo · Luis Ordeñana

¿Qué es PlomerIA?

Conecta clientes con plomeros: el cliente describe su problema, una IA lo diagnostica y le sugiere los mejores profesionales por cercanía y reputación. Pensada para darles la mejor herramienta a las dos partes.

Cliente

- Encuentra un plomero confiable y rápido
- Diagnóstico automático del problema
- Elige día y horario; sigue el estado
- Califica el servicio

Profesional

- Recibe solicitudes de su oficio y zona
- Gestiona agenda y disponibilidad
- Cobra con boleta y suma reputación
- Ve ganancias e historial

Arquitectura: Cliente-Servidor

Estilo cliente-servidor con organización interna en capas

Cliente-Servidor

El cliente (React) solicita recursos/servicios al servidor (FastAPI).

Organización en capas

Routers (API) → Servicios (lógica) → Repositorios (datos) → Modelos.

REST + WebSocket

HTTP para los recursos; WebSocket para el chat en tiempo real.

ORM (SQLAlchemy)

Mapea objetos de Python a las tablas de la base SQLite.

Programación orientada a objetos + SOLID

Pilares de POO

- Encapsulamiento: atributos protegidos
- Herencia: PersonaMixin (datos comunes)
- Polimorfismo: PersonaBase — Usuario y Plomero se tratan igual
- Abstracción: contrato común

SOLID aplicado

- SRP: módulos separados (moderación, calificación, solicitudes)
- LSP: Usuario y Plomero sustituibles por PersonaBase
- OCP: preparado para sumar nuevos oficios sin tocar lo existente

Patrones de diseño

GoF: Singleton, Strategy, Facade · Acceso a datos: Repository, DTO

Singleton

Un único gestor de las salas de chat (WebSocket).

Strategy + Fallback

IA (Gemini) y, si falla, diagnóstico por palabras clave.

Facade / Service Layer

La capa de servicios centraliza y simplifica la lógica.

Repository

Aísla el acceso a datos: se puede cambiar de base sin tocar la lógica.

DTO (Pydantic)

Valida y da forma a los datos de entrada/salida.

Funciones puras

Recomendación y fechas: testeables con pytest.

Metodología: ágil vs. cascada

Trabajamos de forma iterativa, con feedback continuo (Manifiesto Ágil)

Ágil (lo que usamos)

- Entregas incrementales y usables
- Cambios bienvenidos sobre la marcha
- Errores detectados temprano
- Feedback continuo

Cascada (si la usábamos)

- Requerimientos fijos desde el inicio
- Una sola entrega al final
- Feedback recién al cierre
- Cambios costosos y difíciles

Roles del equipo (Scrum)

Pruebas constantes de la app: las hicimos en equipo

María Florencia Iñíguez Trejo

Equipo de Desarrollo

- Desarrollo del backend
- Creación de las tablas de la base
- Pruebas de la API con Swagger
- Inició el tablero de Trello

Aldana Benavent

Product Owner + Dev

- Visión, ideas y mejoras
- Backlog en Trello
- Frontend + ajustes de backend
- Documentación y diagramas
- Cuenta ficticia de plomero (demo de reputación)

Luis Ordeñana

Equipo de Desarrollo

- Comenzó el frontend
- Deploy con Docker (a futuro, a la nube)

Scrum Master: no asignado → faltó facilitación, balance de carga y uso del tablero (lo identificamos como aprendizaje).

Gestión del proyecto y herramientas

Herramientas CASE de apoyo al desarrollo

Proceso

- Trello: tablero de tareas, backlog y prioridades
- Git/GitHub: repositorio, ramas e integración por pull request
- README con instrucciones de clonado y entorno

Herramientas (CASE)

- draw.io / Mermaid: diagramas (UML, flujo, casos de uso)
- Swagger: prueba de la API
- pytest: pruebas unitarias
- Docker: contenerización y deploy

Inconvenientes técnicos y soluciones

Errores detectados al probar la app, y cómo los corregimos

- X Geolocalización por dirección escrita** → se marca la ubicación como punto en el mapa (coordenadas reales)
- X Una solicitud abierta bloqueaba crear otra** → control más estricto de los estados de la solicitud
- X Recomendación no respetaba la regla** → orden por cercanía → puntuación → oficio
- X Tiempos de trabajo incoherentes** → en camino 2 h antes; finalizar 2 h después + boleta
- X Turnos en horarios ya pasados** → solo se ofrecen horarios futuros
- X Se podía finalizar sin boleta** → la boleta es obligatoria para finalizar
- X Duraciones de suspensión invertidas** → groserías 1 mes, cancelaciones 2 meses
- X Calificación incómoda (en historial)** → aparece al finalizar y luego pasa al historial

Un error de diseño que corregimos a tiempo

Refactorizar: reestructurar el código interno sin cambiar lo que el sistema hace

El problema

- Atributos repetidos en distintas clases (Usuario y Plomero): código duplicado
- Rompía el principio DRY y el SRP de SOLID

La solución (refactorización)

- Herencia: centralizamos los datos comunes en una base (PersonaBase)
- Polimorfismo: tratamos a Usuario y Plomero de forma intercambiable (principio de Liskov)

Lo detectamos a tiempo gracias a probar de forma iterativa (ágil) y lo corregimos antes de que escalara.

Problemas de proceso (no técnicos)

Lo que no anticipamos a tiempo en la gestión

1

Sin Scrum Master

No hubo quien facilitara, removiera impedimentos ni balanceara la carga; los roles se mezclaron.

2

Comunicación despareja

El intercambio sobre el proyecto fue fluido en parte del equipo y más acotado en el resto.

3

Trello poco aprovechado

No sostuvimos el uso del tablero — muy ligado a la falta de Scrum Master.

4

Distribución y tiempos

La carga quedó despareja y parte de la integración se resolvió sobre el final.

Aseguramiento de la calidad (QA)

QA = prevención (proceso) · QC = detección (producto)

46

pruebas unitarias
que pasan (pytest)

Cómo cuidamos la calidad

- Pruebas unitarias — caja blanca (pytest): recomendación, fechas, penalizaciones, moderación, boleta
- Pruebas de la API con Swagger durante el desarrollo del backend
- Pruebas manuales — caja negra / exploratorias: uso real como cliente y plomero
- Validaciones del sistema: boleta obligatoria, montos válidos, control de estados y tiempos

Cómo lo dejamos preparado a futuro

- Incorporar nuevos oficios: el código ya quedó preparado para sumar otras actividades (OCP)
- Deploy en la nube con Auto-Deploy, para que no dependa de un servidor propio
- Integración continua (CI): correr las pruebas en cada cambio
- Persistencia de datos (disco persistente o base administrada)
- Refactor de servicios grandes para reforzar SRP
- Editar perfil y otras mejoras de experiencia

Aprendizajes

La IA potencia, pero no reemplaza. Nos dio muchas herramientas y nos ahorró tiempo, pero hay algo insustituible: el razonamiento humano para pensar, decidir y probar. La responsabilidad del código sigue siendo nuestra.

Sin nuestras pruebas manuales constantes, errores como la geolocalización no hubieran salido a la luz.

Probar de forma iterativa (ágil) hizo que el producto creciera y sumara funcionalidades.

Con más atención a los sprints y a la planificación, habríamos distribuido mejor los tiempos.

Demo en vivo

Registro · solicitud · recomendación · aceptación · trabajo y boleta · calificación · y los casos especiales (cancelación, penalización y lenguaje ofensivo).